

Actividad 4: Redes de Datos, Seguridad y Sistemas Distribuidos

Fernando Israel Rios Garcia (AL02877016@tecmlenio.mx),
Artur Mikhail Lazurenko Mannanov (AL03028732@tecmlenio.mx), y
Angel Cuevas Lopez (AL03033114@tecmlenio.mx)

Universidad Tecmlenio

LSTI2308: Sistemas Operativos

Ana Cruz

20 de febrero de 2026

Índice

1. Introducción	4
2. Desarrollo	4
2.1. Parte 1. Diseño y configuración de una red de datos	4
2.1.1. Diseño de la red	4
2.1.2. Configuración de direcciones IP	6
2.1.3. Implementación de servicios de red	7
2.1.4. Implementación de servicios HTTP y HTTPS básicos (Scripts en Python)	7
2.2. Parte 2. Aplicación de medidas de seguridad	8
2.2.1. Configuración de firewalls	8
2.2.2. Implementación de autenticación segura	9
2.2.3. Configuración de HTTPS	10
2.2.4. Análisis de vulnerabilidades y privacidad	10
2.3. Parte 3. Exploración de sistemas operativos distribuidos y P2P	12
2.3.1. Configuración de un sistema de archivos distribuido	12
2.3.2. Exploración del modelo Peer-to-peer	13
2.3.3. Reflexión sobre sistemas distribuidos	13
3. Análisis y reflexión	14
4. Conclusiones	15

Índice de figuras

1. Esquema básico de la red de datos con dos máquinas virtuales y sus IP estáticas.	5
2. Resultados comprobados exitosos de comando ping y visualización de rutas. .	6
3. Accediendo mediante SSH satisfactoriamente al servidor desde el cliente y abriendo su página index inicial HTTP respectiva.	7

4.	Ejecución del servicio HTTP mediante script nativo de Python en el equipo Servidor.	8
5.	Listado de estado activo y reglas restrictivas vigentes en UFW.	9
6.	Fragmento de demostración tras revisión de vulnerabilidades aplicadas a la IP local ejecutando comandos Nmap.	11
7.	Montaje de carpeta y test de interacción al sistema de red transparente de archivos.	13

1. Introducción

El presente reporte detalla el desarrollo de la Actividad 4, que tiene como objetivo principal implementar, configurar y asegurar una red de datos básica, al igual que explorar los sistemas distribuidos mediante la compartición de recursos en un ecosistema virtual. A lo largo de la actividad, se desplegó un entorno compuesto por dos máquinas virtuales Linux conectadas entre sí para observar en la práctica los conceptos del modelo OSI, la configuración manual de direcciones IP estáticas y la habilitación de servicios de red indispensables (como SSH y HTTP).

Posterior a la integración de la red, se aplicaron medidas de seguridad informática estipuladas como estándar en la industria. Esto se logró mediante la configuración de firewalls de host restrictivos con UFW, el uso de criptografía de clave asimétrica para el acceso SSH remoto seguro, la implementación de cifrado HTTPS con certificados autofirmados y el análisis de vectores de posibles vulnerabilidades utilizando herramientas de auditoría perimetral como Nmap. Para terminar, se configuró un sistema de archivos remoto distribuido de red local (NFS) y se exploró de forma análoga el uso práctico de protocolos P2P como BitTorrent para compartir archivos. Integrar los conocimientos de redes, resiliencia y sistemas distribuidos ayuda a consolidar una visión holística sobre la arquitectura tecnológica de las operaciones informáticas actuales.

2. Desarrollo

2.1. Parte 1. Diseño y configuración de una red de datos

2.1.1. *Diseño de la red*

Para la fase de interconexión en esta práctica, se diseñó una red local privada compuesta por dos máquinas virtuales con un sistema operativo derivado de Linux (por ejemplo, Ubuntu Server/Desktop) ejecutándose virtualmente en la misma hipercación principal (VirtualBox o WSL). Dichas entidades se conectaron mediante su abstracción de red virtual interna configurada como `.only-Host` (Solo Anfitrión) para asegurar la comunicación aislada y bidireccional continua:

- **Máquina 1 (Servidor Linux):** Posee la dirección IP estática 192.168.56.10 en su interfaz.
- **Máquina 2 (Cliente Linux):** Posee la dirección IP estática 192.168.56.20 en su interfaz de red.

Figura 1

Esquema básico de la red de datos con dos máquinas virtuales y sus IP estáticas.

El diseño de la comunicación para ambas máquinas en sus intercambios de datos se rige activamente de acuerdo a las capas teóricas del **Modelo OSI** (Tanenbaum & Wetherall, 2011). Esta correlación se puede mapear directamente hacia la red analizada de esta manera:

1. **Capa Física:** Está virtualizada por los adaptadores de red de la máquina hipervisora en VirtualBox, quienes transmiten la señal.
2. **Capa de Enlace de Datos:** Lidera el direccionamiento de hardware; VirtualBox procesa y asocia dinámicamente las direcciones MAC prefiguradas en el switch virtual hacia estas máquinas.
3. **Capa de Red:** Las direcciones IP estáticas locales configuradas (192.168.56.x) proporcionan una ruta lógica a los paquetes ICMP de prueba.
4. **Capa de Transporte:** Uso del protocolo TCP por el cual se conectan los puertos SSH o los paquetes de solicitud HTTP; garantizan una transferencia sin fallos de la petición.
5. **Capa de Sesión:** Administra las sesiones interactivas, como la sesión por terminal persistente obtenida tras el acceso final por SSH.
6. **Capa de Presentación:** Empaqueta los datos con codificación formal para ser transportados; destaca en el cifrado SSL que recubre a las transmisiones asimétricas.

7. Capa de Aplicación: Los clientes e interfaces ejecutadas por el final de la cadena de usuarios, como Firefox o la terminal local de SSH, en adición a demonios en lado del servidor (Apache/SSHD/NFS/P2P).

2.1.2. Configuración de direcciones IP

A fin de asegurar direcciones IP que no cambien al resetear, se modificó el método tradicional de DHCP por parámetros de capa de IP fijos, comúnmente en herramientas modernas de Linux que apuntan a Netplan (/etc/netplan/00-installer-config.yaml) u otros archivos gestores equivalentes:

```
network:
  ethernets:
    enp0s8:
      addresses: [192.168.56.10/24] # Y .20 para la Máquina Cliente en su
      respectivo config
  version: 2
```

Listing 1: Fichero base configuración de IP estática vía terminal (Netplan)

Tras guardar los cambios del registro correspondiente junto al comando `sudo netplan apply`, se comprobó la eficacia de conectividad lógica a nivel red entre ambas computadoras introduciendo instrucciones `ping`. Efectivamente indicando recepción sin latencia perceptible y cero fallas al procesador.

```
# Ejecutado localmente en Máquina Cliente intentando buscar a Máquina
  Servidor
ping -c 4 192.168.56.10
```

Listing 2: Comandos y comprobación simple usando ping

Figura 2

Resultados comprobados exitosos de comando ping y visualización de rutas.

2.1.3. Implementación de servicios de red

El primer equipo virtual asume el rol de distribuidor del servidor web, mientras que el segundo equipo fustigará peticiones cliente tras las integraciones básicas indicadas:

```
sudo apt update
sudo apt install openssh-server apache2 -y
sudo systemctl enable ssh apache2
sudo systemctl start ssh apache2
```

Listing 3: Instrucciones emitidas para instalar paquetes en la unidad Servidor

Ya activos esos protocolos, la Máquina Cliente puede invocar pruebas interactivas de consumo de recursos demostrativas:

- **SSH:** Introducción de `ssh usuario@192.168.56.10` logrando interpelar el prompt remoto.
- **HTTP:** Petición `curl 192.168.56.10` en terminal (o navegación típica desde explorador gráfico si es entorno Desktop), logrando apreciar el HTML renderizado .Apache2 Default Page.^arojado nativamente.

Figura 3

Accediendo mediante SSH satisfactoriamente al servidor desde el cliente y abriendo su página index inicial HTTP respectiva.

2.1.4. Implementación de servicios HTTP y HTTPS básicos (Scripts en Python)

Para cumplir con el requerimiento de levantar servicios web y comprender a fondo la capa de aplicación sin depender de un servidor externo y robusto como *Apache*, se programaron dos scripts en Python (`http_server.py` y `https_server.py`). El lenguaje Python provee un módulo nativo que nos permite levantar un servidor en cuestión de líneas de código.

```
#!/usr/bin/env python3

import http.server
import socketserver
import os

PORT = 80
DIRECTORY = os.path.dirname(os.path.abspath(__file__))

class Handler(http.server.SimpleHTTPRequestHandler):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, directory=DIRECTORY, **kwargs)

with socketserver.TCPServer(("0.0.0.0", PORT), Handler) as httpd:
    print(f"Servidor HTTP activo en el puerto {PORT}. Sirviendo archivos de: {DIRECTORY}")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nServidor HTTP detenido.")
```

Listing 4: Implementación de un servidor HTTP básico en Python (`http_server.py`)

Figura 4

Ejecución del servicio HTTP mediante script nativo de Python en el equipo Servidor.

2.2. Parte 2. Aplicación de medidas de seguridad

2.2.1. Configuración de firewalls

Prevenir intrusiones perimetrales requiere blindar las fronteras operativas expuestas de los servicios (Stallings, 2014). Utilizando el sistema UFW (Uncomplicated Firewall) originado de *iptables*, se generó un ambiente seguro que descarta el tráfico extraño ignorado, abriendo puntualmente solo las vías operacionales indispensables de la infraestructura de este trabajo:


```

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp          # Puerto fundamental del SSH
sudo ufw allow 80/tcp          # Tráfico HTTP en sin encriptar
sudo ufw allow 443/tcp         # Tráfico HTTPS cifrado de Apache
# Puerto NFS reservado y blindado únicamente en dirección al IP Cliente
# específico:
sudo ufw allow from 192.168.56.20 to any port 2049
sudo ufw enable                # Reinicio e iniciación de servicios

```

Listing 5: Adición de reglas de firewall con la utilidad UFW

Figura 5

Listado de estado activo y reglas restrictivas vigentes en UFW.

2.2.2. Implementación de autenticación segura

Habiendo abierto el puerto 22, persistía el riesgo humano de fuerza bruta SSH. Por ende, la política implementada erradicó la validez de contraseñas crudas o convencionales, cediendo su lugar de manera definitiva hacia firmas criptográficas por llaves (Asimetría en la criptografía):

1. Desde la terminal de la Máquina Cliente de origen se corrió el comando: `ssh-keygen -t rsa -b 4096`, generando así un par combinatorio secreto para el anfitrión.
2. Posteriormente, el contenido exacto de ese candado público (`.pub`) se instaló a distancia gracias a `ssh-copy-id usuario@192.168.56.10`.
3. Evaluada la conexión inicial que ignoraba credenciales textuales, se bloqueó tajantemente a todo tercero mediante la edición nativa del archivo maestro `/etc/ssh/sshd_config` al imponer forzosamente el parámetro final en `PasswordAuthentication no`.
4. Finalmente se acató una recarga con `sudo systemctl restart ssh`.

2.2.3. Configuración de HTTPS

Se introdujo cifrado completo punto a punto dentro de la capa de presentación / transporte web utilizando bibliotecas como la proveída por la suite **OpenSSL** interactuando con las librerías nativas de Python. Como la red de validación fue interna y de índole local (es decir, inaccesible por los canales mundiales certificados reales de Let's Encrypt), se aprovisionaron claves y certificados autofirmados (Self-Signed Certificates) que son consumidos directamente por el script `https_server.py` que eleva un `ssl.wrap_socket` sobre el protocolo web:

```
# Creación del binomio público/privado con expira a largo plazo (server.
pem)
openssl req -new -x509 -keyout server.pem -out server.pem -days 365 -nodes
-subj '/CN=localhost'

# Iniciar servidor seguro HTTPS en Python
sudo python3 https_server.py
```

Listing 6: Proceso de creación SSL y ejecución de HTTPS nativo en Python

El procedimiento culminó al visitar con un cliente la URL encriptada `https://192.168.56.10` y al probar una conexión directa con `curl -k`, evidenciando un canal de comunicación web encapsulado a nivel TLS robusto pero advertido en el navegador de no poseer validación raíz comercial.

2.2.4. Análisis de vulnerabilidades y privacidad

Validar las topologías requiere auditorio informático y monitoreo pasivo. Consiguientemente, escaneamos remotamente de Máquina a Máquina todos los puertos abiertos en estado general. Utilizamos el ejecutable **Nmap** para una traza: `nmap -sV -A 192.168.56.10`. Tras analizar reportes de Nmap (y con Lynis de revisión nativa), surgen 3 riesgos/vulnerabilidades de particular atención a la privacidad corporativa:

- 1. Vulnerabilidad (Exposición no regulada y huella en capas web):

Usualmente Apache puede desplegar un encabezado conteniendo las versiones internas de compilación u OS a la vista de extraños, induciendo explotación 0-day si un bug se revela en dicha versión.

Solución: Implementar ofuscación desactivando y escondiendo las firmas del Server Header en `apache2.conf` (`ServerSignature Off` y `ServerTokens Prod`).

■ 2. Vulnerabilidad (Sobreuso de peticiones - Denegación DDOS

HTTP/SSH): Nuestro entorno de acceso remoto está publicado correctamente, mas es vulnerable a inundación abusiva volumétrica que acapare CPU (Ataque DDos) impidiéndonos operar nuestro propio equipo.

Solución: Complementar al firewall habilitado de Linux con una herramienta antispam como `fail2ban`, bloqueando al instante en Netfilter (Iptables/UFW) la procedencia IP de quién cause colapsos con intentos repetitivos excesivos o tráfico anómalo.

■ 3. Riesgo de Intrusión Intermedia (Man In The Middle - MitM):

Como se advirtió, los certificados emitidos al HTTPS local carecen de validador universal central; son autofirmados. Alguien interno en la oficina o red local manipulando envenenamientos con protocolos de resolución (ARP) pudiese clonar la validación e interceptar comunicaciones engañando al usuario incauto.

Solución: Distribuir certificados de autoridades corporativas validados u obtenidos verídicamente de agencias de confianza global (*Let's Encrypt*) para asegurar inviolabilidad perimetral total confirmada del cifrado de túnel HTTPS local.

Figura 6

Fragmento de demostración tras revisión de vulnerabilidades aplicadas a la IP local ejecutando comandos Nmap.

2.3. Parte 3. Exploración de sistemas operativos distribuidos y P2P

2.3.1. Configuración de un sistema de archivos distribuido

Un sistema distribuido exitoso se encarga de homologar entornos. Como preámbulo funcional local, aplicamos **NFS** (Network File System) de manera que Máquina Servidora brinde cuotas nativas crudas que emulen el disco C: de un Cliente con una carpeta de sincronía completa bidireccional (/var/nfs_share).

Ejecución nativa en el Host-Servidor compartiendo el material:

```
sudo apt install nfs-kernel-server
sudo mkdir -p /var/nfs_share
sudo chown nobody:nogroup /var/nfs_share/      # Evitar denegación de
    lectura a ajenos
sudo nano /etc/exports
# Agregar línea exacta validando UFW previa:
# /var/nfs_share      192.168.56.20(rw, sync, no_subtree_check)
sudo systemctl restart nfs-kernel-server
```

Listing 7: Habilitar daemon base exportador de archivos NFS

Sincronización natural interna aplicada en terminal Cliente:

```
sudo apt install nfs-common
sudo mkdir -p /mnt/nfs_clientshare
sudo mount 192.168.56.10:/var/nfs_share /mnt/nfs_clientshare
```

Listing 8: Montaje directo de recursos foráneos mediante utilidades de red base (NFS Client)

Luego de concluir con efectividad el ensamble remoto, es posible probar cómo un documento creado desde Máquina 2 (usando el comando local `touch /mnt/nfs_clientshare/textonfs.txt`) se halla ya presente de facto explorando /var/nfs_share/ en el equipo inicial de la habitación host. Esta experimentación confirma sólidamente la característica de **transparencia de ubicación y acceso**; sin lugar a dudas el cliente gestiona archivos como locales (ext4) y está inconsciente de los procesos de

empaquetamiento y encapsulación cruzada que enmascaran el hecho irrefutable de que estos yacen alejados físicamente subastados mediante tramas en red a un disco duro remoto en otro servidor ajeno de fondo.

Figura 7

Montaje de carpeta y test de interacción al sistema de red transparente de archivos.

2.3.2. *Exploración del modelo Peer-to-peer*

De vuelta a las implementaciones centralizadas (SSH y NFS donde Servidor guarda todo), surge otro arquetipo colosal **Peer-to-Peer** (P2P). Su naturaleza elimina intermediarios estelares promoviendo clientes que simultáneamente sean servidores autosuficientes entre sí repartiendo fragmentos del bloque principal porciones pequeñas (peers). Usamos e instalamos en la dupla la tecnología *Transmission-cli* o *qBittorrent*.

Se forjó nativamente un archivo pesado local en Máquina 1 llamado hipotéticamente *datos_distribuidos.iso* a partir de allí se compila el .archivo enlace conocido como .torrent. Al transferir este hash encriptado de enlace torrent a la Máquina 2 consumidora (empleando el cliente de su respectivo software equivalente), esta contacta de inmediato al servidor para solicitar subida fragmentada al instante. En un esquema natural con centenares de clientes interconectados mediante enjambres "torrents", cada peer enjambre provee su ínfima porción descargada ayudando a todos. La red P2P exalta descentralización, donde cada máquina virtual o equipo local provee de redundancia inamovible para salvar la integridad subida al colectivo, previniendo fallos críticos de que la infraestructura primaria en Máquina 1 reciba daño y colapsen las bajadas de los miles de peers restantes.

2.3.3. *Reflexión sobre sistemas distribuidos*

Nuestra labor ejecutando NFS y BitTorrent arroja indicios sustanciales que demuestran qué capacidad alberga toda red de volverse entidades integrales para fines colaborativos. Observamos como parte de sus **ventajas**: primero, un enorme abaratamiento

y homogeneización; los empleados de un complejo pueden interactuar sobre reportes con material consolidado como local pese a provenir de almacenaje externo (NFS). Segundo, en sistemas distribuidos verdaderos (P2P) se incrementa de forma exponencial la redundancia y resiliencia corporativa anti-censura frente a una falla abrupta de hardware de un único servidor principal.

Sin embargo, en el apartado de sus naturales **desventajas**: encontramos en repetidas ocasiones que el balance de seguridad escala con la complejidad. Un puerto expuesto al escaneo público indebidamente para compartición de fichas es devastador, pues el control descentralizado no siempre asegura que el receptor remoto cumpla los criterios de pureza y limpieza en el software original, o incluso a nivel hardware, produce una severa fatiga saturante sobre las interfaces de red locales por el exceso agresivo de descubrimiento por difusión de paquetes e intercambio por fragmentos P2P si uno carece del monitoreo de tráfico con Quality of Service (QoS) aptos.

3. Análisis y reflexión

El ensamble final integrando conceptos pragmáticos del curso logró trazar en el plano material los hilos comunicativos conceptuales correspondientes al modelo puramente teórico (OSI). Afrontamos con efectividad retos al diagnosticar tempranamente fallas en sintaxis manual de certificados de encriptación autofirmados al interactuar su instalación con los sub-demonios de Apache SSL y se reaccionó acorde a los errores imprevistos en los montajes de las particiones NFS en entornos virtualizados Linux.

El corolario del progreso evidenció cómo la seguridad computacional interconectada carece de soluciones monolíticas mágicas o absolutas; por ende, requiere esquematizarse con engranajes redundantes formados por candados asimétricos, filtros rudimentarios estrictamente acotados (UFW) y ocultamientos del vector físico expuesto hacia clientes o hacia enjambres distribuidos.

4. Conclusiones

El recorrido y desglose progresivo del presente reporte atestigua indudablemente un hecho pragmático: el conocimiento de redes de comunicaciones no es trivial, y se entrelaza estrechamente con el blindaje seguro y el análisis logístico en su aplicación empresarial diaria. Es vital reconocer a todo esquema de autenticación como inerte per se frente a escalamientos y fisuras, puesto que ignorar directrices como un canal encriptado moderno o dejar abierto de forma pasiva fallas triviales detectadas en un reporte local arroja brechas para intrusos y ransomware.

Sintetizando conclusiones, los protocolos y despliegues virtualizados del paradigma de Sistemas Operativos Distribuidos y sus implementaciones de pares P2P demuestran ser innovaciones eficientes, fiables y transparentes destinadas fundamentalmente para equilibrar latencias con descentralización de recursos y maximizar capacidades de usuarios aislados; no obstante, todo esto debe acoplarse y subyugarse siempre al correcto monitoreo preventivo de un control de flujos auditable capaz y con las salvaguardas restrictivas competentes para asegurar su persistencia estable e ininterrumpible.

Referencias

Stallings, W. (2014). *Comunicaciones y Redes de Computadores* (10.^a ed.). Pearson.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Redes de computadoras* (5.^a ed.). Pearson Educación.

Universidad Tecmilenio. (s.f.). *Actividad 4: Redes de Datos, Seguridad y Sistemas*

Distribuidos. Consultado el 17 de febrero de 2026, desde

<https://utm-cdn-labcontenidos->

[htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-](https://utm-cdn-labcontenidos-htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-)

[v1/actividad-04.html?version=1](https://utm-cdn-labcontenidos-htfaarehf2gcfycs.a01.azurefd.net/contenido/profesional/lsti2308/entregables-v1/actividad-04.html?version=1)